

Local AI Platform — What It Is, In Plain Language

The one-sentence version

Local AI Platform makes a self-hosted AI coding assistant that actually understands *your* codebase — running entirely on your own hardware, with no code ever leaving your building.

The problem it solves

AI coding assistants like GitHub Copilot or Cursor are useful, but they send your code to a cloud service (OpenAI, Anthropic, and others) to work. For a lot of teams that is a non-starter:

- **Regulated or sensitive code** — finance, healthcare, defense, or anyone with strict IP rules often cannot legally or contractually send source code to a third-party cloud.
- **Cost** — heavy use of cloud AI coding tools bills per token and adds up fast across a team.
- **Generic answers** — a cloud model knows programming in general, but not *your* project. It hasn't read your modules, your naming conventions, your internal APIs.

Local AI Platform addresses all three by running the whole thing locally and making the assistant aware of the specific repository it is working on.

How it works, without jargon

Coding assistants (like Cline or Claude Code) normally talk directly to an AI model. This project inserts a smart layer in between — a "gateway" — that the assistant talks to instead.

Before passing a request along to the local AI model, the gateway:

1. **Reads and indexes your codebase** so it knows what functions, classes, and modules exist.
2. **Figures out what the developer is actually asking** (explaining code, fixing a bug, adding a feature) and pulls in the *most relevant* pieces of your code as context.
3. **Avoids repeating itself** — it does not re-send code the assistant has already seen in the conversation.
4. **Controls how much it sends** so responses stay fast, since bigger requests are slower.
5. **Keeps everything compatible** with the standard AI interface, so existing tools work unchanged — you just point them at the gateway instead of the cloud.

The result: an assistant that answers with knowledge of *your* actual code, runs on hardware you control, and never sends a line of source anywhere.

What works today (not vaporware)

These are built, running, and measured on real usage:

- The gateway, end to end, with a real self-hosted model (vLLM backend).
- Codebase indexing and relevant-context injection — the core differentiator.
- Intent detection (explain / debug / implement) driving what context gets pulled.
- "Delta" injection that avoids re-sending code already in the conversation.
- Two independent controls over request size (repository context budget, conversation history cap) — the

- levers that trade response speed against how much context the model sees.
- Full measurement tooling: every request is logged and analyzable for latency, token cost, and context behavior.

Measured, not claimed: on the project's own retrieval benchmark, relevant-context injection roughly doubled answer relevance (6/14 → 12/14), and the size controls measurably reduce response latency on heavy requests.

What is planned (and where help is wanted)

The repository contains substantial scaffolding for larger capabilities that are **not yet wired into the live system** — deliberately kept separate until they are ready. These are the future: autonomous multi-step engineering, richer code-modification tooling, deeper analysis capabilities. They are the clearest opportunities for contributors who want to own a meaningful, self-contained piece. See `CONTRIBUTING.md`.

What this project needs

This is a solo build that has reached a working, honest, measurable state. To grow it needs **people with time to code and analyze** — not funding. The most valuable help is in wiring up the planned capabilities, hardening the core, and validating quality on real-world use.

If you want an AI coding assistant that respects your code's privacy and actually knows your codebase — and you can write Python — this is a project where one person's contribution moves the needle.